

01. 카프카 기본기

카프카

#ApacheKafka

#카프카

#apache

#Event

#Producer

#Consumer

#Broker

#Cluster

#Topic

#Partition

#Offset

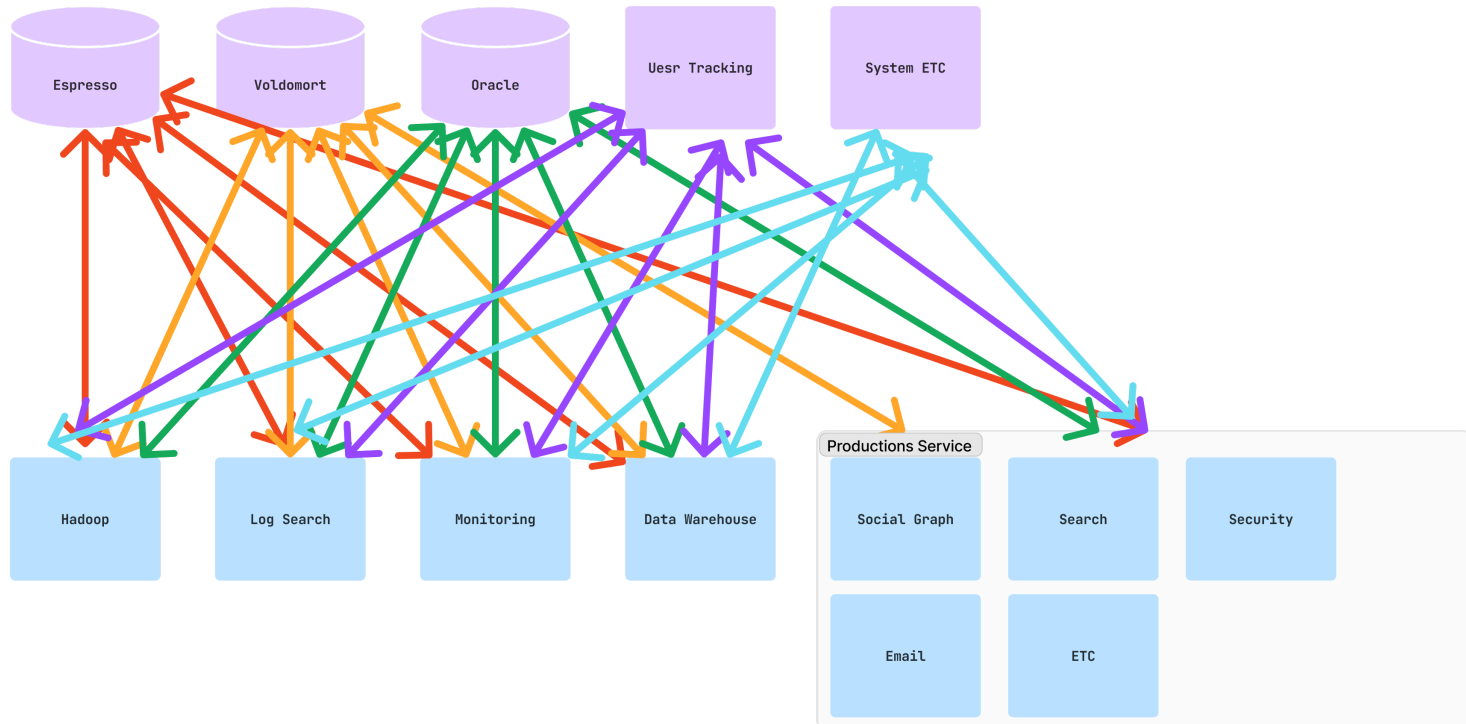
#ConsumerGroup

카프카란?

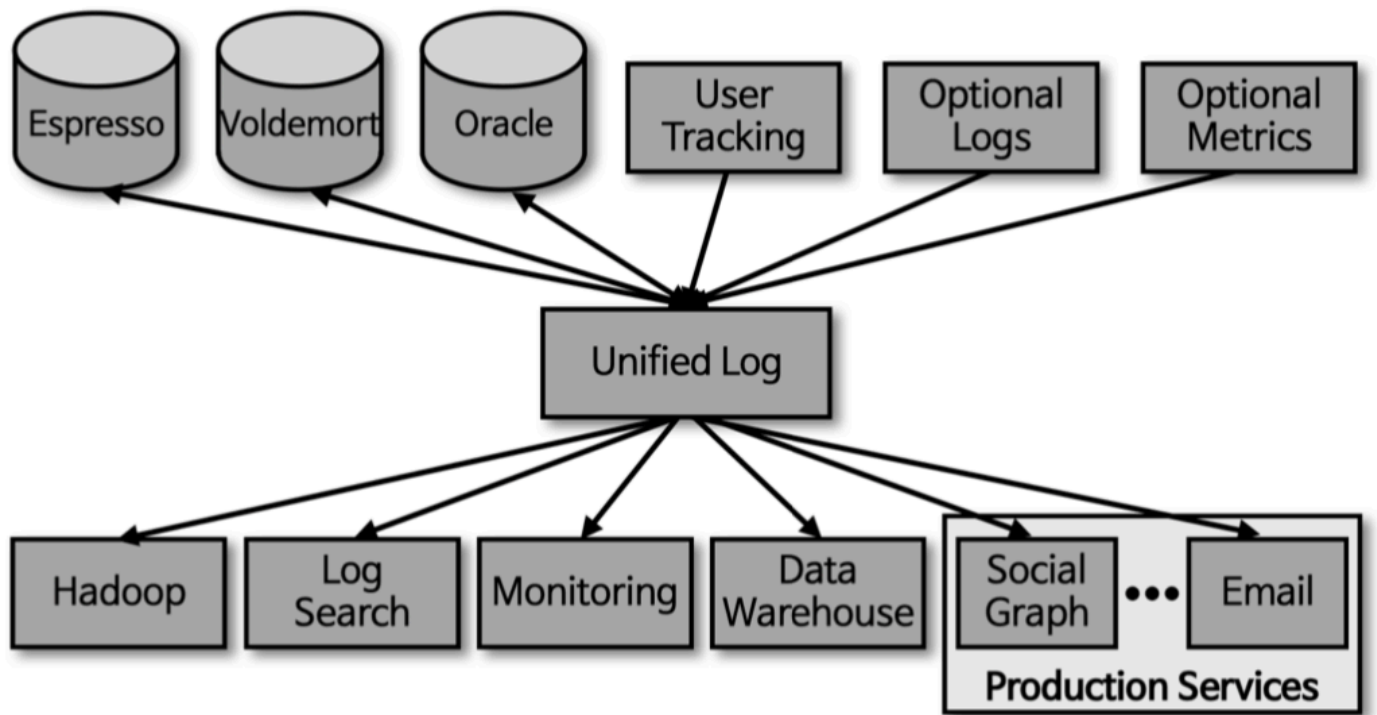
- 아파치 카프카는 대용량의 실시간 데이터 스트림을 안정적으로 처리하기 위한 **분산 이벤트 스트리밍 플랫폼**(*Distributed Event Streaming Platform*) 입니다.
- 단순한 메세지 큐를 넘어, 이벤트(*Data*)를 발행(*publish*) 하고, 구독(*Subscribe*) 하여, 발생한 이벤트를 저장(*Store*)하고, 처리(*Process*) 하는 모든 기능을 제공합니다.
 - 수 많은 기업에서 데이터 파이프라인, 실시간 분석, 마이크로서비스 간 통신 등 다양한 목적에서 사용하고 있다.

카프카 사용 이전

- 도입 이전 상태



- 도입 이후



카프카 개념 및 핵심 요소

요소	설명
Event (이벤트)	Kafka에서 주고받는 데이터의 기본 단위입니다. '메시지' 또는 '레코드'라고도 불리며, Key, Value, Timestamp, Metadata 등으로 구성됩니다.
Producer (프로듀서)	이벤트(메시지)를 생성하여 Kafka에 보내는 역할 을 하는 클라이언트 애플리케이션 입니다.
Consumer (컨슈머)	Kafka로부터 이벤트(메시지)를 가져와서 사용하는 역할을 하는 클라이언트 애플리케이션입니다.
Broker (브로커)	Kafka 서버 자체 를 의미합니다. 프로듀서로부터 받은 이벤트를 디스크에 안정적으로 저장 하고, 컨슈머가 요청할 때 이벤트를 전달 합니다.
Cluster (클러스터)	여러 대의 브로커 로 구성된 그룹입니다. 클러스터를 통해 높은 처리량과 데이터의 안정성(내결함성, Fault Tolerance)을 확보합니다.
Topic (토픽)	이벤트가 저장되는 분류 단위 입니다. 파일 시스템의 폴더나 데이터베이스의 테이블과 유사하며, 프로듀서는 특정 토픽에 메시지를 보내고 컨슈머는 특정 토픽의 메시지를 구독합니다.
Partition (파티션)	하나의 토픽을 여러 개로 나누어 저장하는 단위 입니다. 파티션을 통해 하나의 토픽이 감당할 수 있는 데이터의 양을 늘리고, 병렬 처리 를 가능하게 합니다. 메시지 순서는 각 파티션 내에서만 보장 됩니다.
Offset (오프셋)	각 파티션 내에서 메시지가 가지는 고유한 순번(ID) 입니다. 컨슈머는 이 오프셋을 기준으로 어디까지 메시지를 읽었는지 추적 합니다.
Consumer Group (컨슈머 그룹)	하나의 토픽을 여러 컨슈머가 병렬로 처리 하기 위해 사용하는 그룹 단위 입니다. 하나의 파티션은 컨슈머 그룹 내에서 단 하나의 컨슈머에게만 할당되므로, 컨슈머의 수만큼 병렬 처리 성능을 높일 수 있습니다.

특징 및 기능

높은 처리량 (*High Throughput*)

- 디스크에 순차적으로 로그 기록하고, OS의 페이지 캐시와 제로 카피(*Zero-Copy*) 기술을 활용하여 대용량의 데이터를 매우 빠르게 처리

확장성 (*Scalability*)

- 데이터가 많아지면 브로커와 파티션을 추가하는 수평 확장(*Scale-out*) 이 매우 용이함.

영속성 및 내결함성 (*Durability & Fault Tolerance*)

- 수신한 데이터를 메모리가 아닌 디스크에 저장하여 영속성 보장
- *Replication*(복제) 기능을 통해 여러 브로커에 데이터를 복제해두므로, 일부 브로커에 장애가 발생해도 데이터 유실 없이 안정적인 서비스 가능.

시스템 간 디커플링 (*Decoupling*)

- 프로듀서와 컨슈머가 서로의 존재를 알 필요 없이 오직 Kafka 와 통신합니다.
- 시스템 간의 *의존성이 크게 낮아져 유연하고 확장 가능한 아키텍처를 구축*할 수 있습니다.
- *이벤트 아이디어를 시스템 전체 관점으로 확장* 하는 원리

장단점

Advantages (장점)

- **대용량 실시간 처리**
 - 초당 수십만 건 이상 메시지를 안정적으로 처리할 수 있어 빅데이터 스트리밍에 최적화
- **유연한 아키텍처 구축**
 - 시스템 간의 강한 결합(*Coupling*) 을 해소하여, 각 시스템이 독립적으로 발전하고 확장될 수 있는 *마이크로서비스 아키텍처*에 매우 적합
- **데이터 파이프라인 통합**
 - 다양한 데이터 소스와 연결하는 중앙 허브 역할을 수행하여, 데이터 파이프라인을 단순화하고 통합 관리할 수 있습니다.
- **다양한 생태계**
 - *Kafka Connect*(데이터 연동) , *Kafka Streams*(실시간 처리) 등 강력한 서브 프로젝트와 풍부한 커뮤니티를 보유하고 있습니다.

Disadvantages (단점)

- **높은 운영 복잡도**
 - 브로커, 주키퍼 등 *직접 설치하고 운영해야 할 요소가 많아 초기 설정과 유지보수에 대한 학습 곡선*이 가파르다.

- 메시지 순서의 제약
 - 메시지의 순서는 토픽 전체가 아닌 파티션 단위로만 보장 됨.
 - 전체 메시지의 순서 보장이 필요한 경우, 키를 이용한 파티셔닝 전략 등 추가적인 설계가 필요합니다.
- 브로커 튜닝의 어려움
 - 최적의 성능을 내기 위해서는 시스템의 특성에 맞게 브로커의 다양한 옵션을 세밀하게 튜닝해야 하는 경우 많음.
- 실시간성 보장은 아님.
 - 실시간에 가까운 플랫폼.
 - 네트워크 지연이나 브로커 상황에 따라 수 밀리초(ms) 에서 수십 밀리초 지연 (latency) 발생 할 수 있다.

서버 간 **Kafka** 통신 구상 단계

- 서버 간 통신을 구상할 때 다음의 4 단계를 따르는 것이 일반적

1. 이벤트(#Event) 정의
2. 토픽(#Topic) 설계
3. 프로듀서(#Producer) 구현
4. 컨슈머(#Consumer) 구현

1. 이벤트 정의

- 어떤 동작이나 상태 변화를 메시지로 만들 것인지 결정합니다.

예시

사용자가 회원가입을 완료했다.
주문이 결제되었다.
상품 재고가 변경되었다.

2. 토픽 설계

- 정의한 이벤트를 어떤 채널 (Channel) 또는 주제 (Subject) 에 담을지 설계합니다.

예시

user-signup → 사용자가 회원가입 완료했습니다.
order-payment-completed → 주문 결제되었다.
product-stock-changed → 상품 재고 변경

3. 프로듀서 구현

- 이벤트를 발생시키는 서버에 카프카로 메시지를 보내는 로직을 구현합니다.

예시

```
await 회원가입_트랜잭션_처리();
await 토픽전송("user-signup", { 회원 정보 객체 });
```

4. 컨슈머 구현

- 해당 이벤트를 받아 처리해야 하는 모든 서버에 메시지를 구독하는 로직을 구현합니다.

예시

```
// 메일 서버 구독
const mailServer = kafka.컨슈머();
mailServer.토픽.구독("user-signup", async (data : any) => {
  // 3. 프로듀서에서 회원정보를 가입 축하를 위한 로직이 실행
})

// 분석 서버
const analysisServer = kafka.컨슈머();
analysisServer.토픽.구독("user-signup", async (data : any) => {
  // 3. 회원가입을 했으니 신규 회원에 대한 분석 서버 가동
})
```

대표적인 통신 패턴

1:1 기본패턴 통신

가장 기본적인 형태로, 특정 서버가 발생시킨 이벤트를 다른 특정 서버가 받아 처리하는 구조.

```
const 시나리오 = {
  설명 : "`주문 서버`에서 주문이 생성되면 `배송 서버`가 이를 인지하고 배송 준비를 시작합니다.",
  구상 : [
    "1. 이벤트 : 주문 생성 완료",
  ],
}
```

```
"2. 토픽 : order-created",
"3. 프로듀서" : "주문 서버",
"4. 컨슈머" : "배송 서버"
}
```



팬아웃 (Fan-out) 패턴 1:N 통신

하나의 이벤트가 발생했을 때, 여러 서버가 각기 다른 작업을 동시에 처리하는 가장 일반적인 Kafka 사용 패턴입니다.

시스템 간의 의존성을 완벽하게 분리(디커플링)하는 Kafka의 장점이 가장 잘 드러납니다.

```
const 시나리오 = {
  설명 : "`사용자 서버`에서 신규 회원가입 이벤트가 발생하면, 여러 서버가 이 정보를 받아 각각의 임무를 수행합니다.",
  구상 : [
    "1. 이벤트 : 신규 회원 가입",
    "2. 토픽 : user-signed-up",
    "3. 프로듀서" : "사용자 서버",
    "4. 컨슈머" : [
      "메일 서버",
      "포인트 서버",
      "데이터 분석 서버"
    ]
  ]
}
```

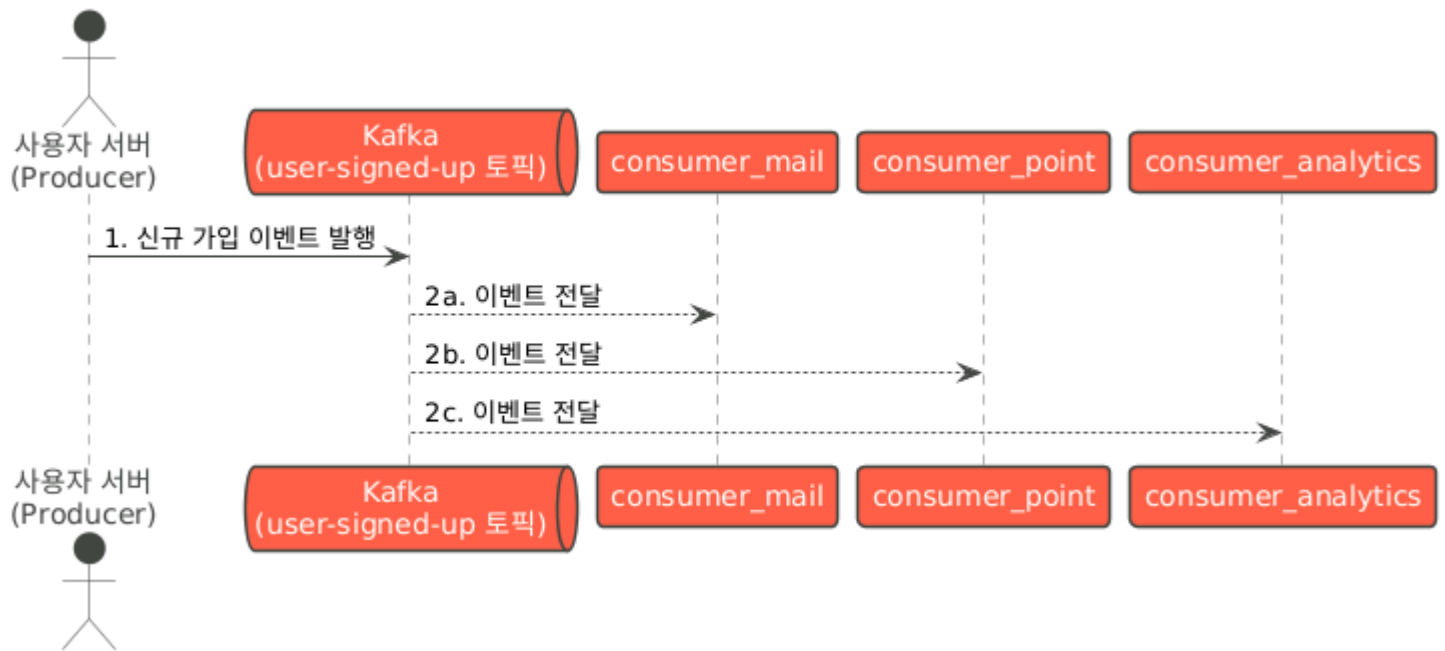
- 메일 서버 : 환영 메일 발송
- 포인트 서버 : 신규 가입 포인트 지급
- 데이터 분석 서버 : 신규 가입자 통계 집계

이 구조에서 **사용자 서버**는 메일, 포인트, 분석 서버의 존재를 전혀 알 필요가 없습니다.

그저 "회원가입이 일어났다"는 사실을 Kafka에 알리기만 하면 됩니다.

나중에 "추천 친구 목록 생성 서버"가 추가되더라도 **사용자 서버**의 코드는 단 한 줄도 변경할 필요 없이 새로운 서버가 토픽을 구독하기만 하면 됩니다.

이처럼 Kafka를 중심으로 통신을 구상하면 각 서버가 독립적으로 확장되고 유지보수될 수 있는 **유연하고 확장성 높은 시스템**을 만들 수 있습니다.



- 하나의 **producer** 가 보낸 메시지를 중앙의 **kafka_topic** 이 받습니다.
- **kafka_topic** 에서 여러 **consumer** 로 화살표가 각각 뻗어 나가며 1:N 통신을 명확하게 보여줍니다.

주키퍼

- 분산 시스템에서 여러 서버들의 상태를 관리하고 작업을 조율하는 **코디네이션 서비스**.
- 여러 서버가 하나의 시스템 처럼 동작하게 만드는 **교통 정리 경찰** 또는 **오케스트라의 지휘자** 라고 생각하면 된다.

역할과 중요성

- 분산 환경에서 발생하는 복잡한 문제를 해결하기 위한 매우 중요한 역할을 수행함.

클러스터 메타데이터 관리

분산 시스템에 참여하는 서버(Node)들의 설정 정보, 상태 등 **중요한 운영 데이터(Metadata)**를 한곳에서 통합 관리함.

- 어떤 서버가 동작 중이고, 어떤 서버에 장애가 발생했는지, 데이터는 어디에 저장되어있는지등 모든 서버가 동일하게 인지할 수 있게 함.
- 이 **정보가 일치하지 않으면 시스템 전체가 오작동** 할 수 있음.

리더 선출 (Leader Election)

여러 서버 중 하나의 서버를 리더(Leader) 로 선출하는 역할을 합니다.

리더는 쓰기 작업이나, 특정 연산을 책임지며, 데이터의 일관성을 유지하는 중심점 역할을 함.

- 모든 서버가 동시에 데이터를 변경하려고 하면 충돌이 발생하지만, 리더를 선출하면 데이터 변경 작업을 한곳으로 집중시켜, 데이터 정합성을 보장할 수 있다.

- 리더 서버에 장애가 발생하면 즉시 다른 서버를 새로운 리더로 선출 하여 서비스 중단을 최소화 함.

분산 동기화 및 락 (LOCK)

여러 서버가 동시에 같은 데이터나 자원에 접근하지 못하도록 막는 분산 락(*Distributed Lock*) 기능을 제공함.

- 특정 작업은 반드시 한번에 하나의 서버만 수행해야 합니다. 예시로, 은행의 출금 작업이 동시에 여러 번 처리되면 안되는 것과 같다.
- 주키퍼는 이러한 동시성 문제를 해결하여 작업의 원자성을 보장함.

상태 감지 및 알림

특정 데이터나 서버의 상태의 변경을 감지하고, 이를 구독하고 있는 다른 서버들에게 즉시 알려주는 감시(*Watch*) 기능을 제공.

- 클러스터에 새로운 서버가 추가되거나 기존 서버에 장애가 발생했을 때, 이 변경 사항을 모든 관련 서버가 신속하게 인지하고 대응할 수 있도록 함.

카프카와 주키퍼

과거 카프카는 주키퍼에 매우 크게 의존했습니다.

카프카 클러스터의 모든 정보(*브로커 ID, 토픽 정보, 파티션 리더 등*)를 주키퍼에 저장하고 관리했습니다.

카프카 브로커를 실행하려면 반드시 주키퍼가 먼저 실행되어 있어야 했습니다.

하지만 이 구조는 다음과 같은 단점이 있음.

- **운영 복잡성**: 카프카와 주키퍼라는 두 개의 분산 시스템을 모두 관리해야 함.
- **확장성 한계**: 주키퍼의 성능이 카프카 클러스터 전체의 확장성을 제한하는 병목 지점이 되기도 함.

최신 동향: 주키퍼 없는 카프카 (KRaft 모드)

이러한 의존성을 없애기 위해 최신 카프카 버전(2.8 이상)에서는 **KRaft(Kafka Raft)** 모드가 도입. KRaft 모드는 주키퍼가 하던 메타데이터 관리와 리더 선출 역할을 카프카 클러스터 자체가 직접 수행.

결론적으로, 주키퍼는 분산 시스템의 안정성과 데이터 일관성을 보장하는 데 필수적인 핵심 서비스 이다.

비록 최신 카프카는 KRaft 모드를 통해 주키퍼 의존성을 제거하고 있지만, 주키퍼가 해결하고자 했던 문제(코디네이션, 리더 선출, 동기화)는 여전히 분산 시스템에서 가장 중요한 과제이며, 주키퍼의 원리를 이해하는 것은 분산 아키텍처를 이해하는 데 매우 중요합니다.

Kafkajs

```
npm i kafkajs
```

- **Example**


```

@InjectableView()
export class KafkaService implements OnModuleInit, OnModuleDestroy {
  private kafka: Kafka;
  private producer: Producer;
  private consumer: Consumer;

  constructor(private readonly groupId: string) {
    this.kafka = new Kafka({
      clientId: KAFKA_CONFIG.clientId,
      brokers: KAFKA_CONFIG.brokers,
    });

    this.producer = this.kafka.producer();
    this.consumer = this.kafka.consumer({ groupId });
  }

  async onModuleInit() {
    await this.producer.connect();
    await this.consumer.connect();
  }

  async onModuleDestroy() {
    await this.producer.disconnect();
    await this.consumer.disconnect();
  }

  async sendMessage(topic: string, message: any) {
    await this.producer.send({
      topic,
      messages: [
        {
          value: JSON.stringify(message),
          timestamp: Date.now().toString()
        }
      ]
    });
  }

  async subscribeToTopic(topic: string, callback: (message: any) => void) {
    await this.consumer.subscribe({ topic, fromBeginning: false });

    await this.consumer.run({
      eachMessage: async ({ topic, partition, message }) => {
        const messageValue = message.value?.toString();
        if (messageValue) {
          const parsedMessage = JSON.parse(messageValue);
          callback(parsedMessage);
        }
      },
    });
  }
}

```

- `clientId` : 어떤 서버인지 파악하는 용도
- `brokers` : 클러스터가 3개 이면, 3개 의 배열로 적으면 된다.

```
@Post()
async createUser(@Body() userData: any) {
  // Gateway에서 User 서비스로 메시지 전송
  await this.kafkaService.sendMessage(KAFKA_CONFIG.topics.USER_EVENTS, {
    action: 'CREATE_USER',
    data: userData,
    timestamp: new Date().toISOString()
  });

  return { message: 'User creation request sent' };
}
```

```
@Injectable()
export class UserService implements OnModuleInit {
  constructor(private readonly kafkaService: KafkaService) {}

  async onModuleInit() {
    // User 이벤트 구독
    await this.kafkaService.subscribeToTopic(
      KAFKA_CONFIG.topics.USER_EVENTS,
      this.handleUserEvent.bind(this)
    );
  }

  private async handleUserEvent(message: any) {
    console.log('User Service received message:', message);

    switch (message.action) {
      case 'CREATE_USER':
        await this.createUser(message.data);
        break;
      case 'GET_USER':
        await this.getUser(message.data.id);
        break;
      default:
        console.log('Unknown action:', message.action);
    }
  }

  private async createUser(userData: any) {
    // 사용자 생성 로직
    console.log('Creating user:', userData);

    // Log 서버로 로그 메시지 전송
    await this.kafkaService.sendMessage(KAFKA_CONFIG.topics.LOG_EVENTS, {
      service: 'user-service',
      action: 'USER_CREATED',
      data: userData,
    });
  }
}
```

```
        timestamp: new Date().toISOString(),
        level: 'INFO'
    });
}

private async getUser(userId: string) {
    // 사용자 조회 로직
    console.log('Getting user:', userId);

    // Log 서버로 로그 메시지 전송
    await this.kafkaService.sendMessage(KAFKA_CONFIG.topics.LOG_EVENTS, {
        service: 'user-service',
        action: 'USER_RETRIEVED',
        data: { userId },
        timestamp: new Date().toISOString(),
        level: 'INFO'
    });
}
}
```